Stacks: Last In, First Out

Programming and Data Structures — Week 3, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain Last In, First Out (LIFO) ordering and identify it operating in at least three everyday situations outside of computing; implement a `Stack` class backed by a Python list, exposing `push`, `pop`, `peek`, and `is_empty`; state the time complexity of every stack operation and justify each cost directly from Week 2's analysis of array behavior, rather than as a new unexplained fact; and trace a balanced-parentheses checker executing on a real input string, one character at a time.

Outline

Today covers a plate-stacker analogy that gives LIFO a physical anchor; a precise definition of Last In, First Out ordering; building a `Stack` class directly on top of the dynamic array analyzed in Week 2; the cost of every stack operation, justified from that same analysis rather than presented as a new fact to memorize; the call stack, a stack every student has already been using implicitly since Week 1's first function call; a full worked trace of a balanced-parentheses checker; an in-class quiz; and a summary.

The plate-stacker analogy

Picture a spring-loaded cafeteria plate dispenser: every new plate is placed on top of the stack, and every plate removed is taken from the top. There is no way to pull a plate from the middle or the bottom without first removing everything stacked above it. The last plate placed is always the first one removed. This single physical constraint — you may only ever touch the top — is the complete definition of a stack as a data structure; everything else in this lecture is a consequence of enforcing that one rule in code.

Defining LIFO precisely

LIFO stands for Last In, First Out. A stack, as an abstract data type, supports exactly two operations that change its contents: `push(x)`, which places a new element on top, and `pop()`, which removes and returns whatever element currently sits on top. Every other operation typically offered — `peek()`, which looks at the top element without removing it, and `is_empty()`, which reports whether any elements remain — only reads the stack's state and never changes the order of its contents.

Other everyday LIFO examples

LIFO ordering shows up constantly outside of plate dispensers. A browser's back button returns you to the most recently visited page first — the "last in," the page you just came from, is the "first out" when you press back. An undo command reverses the most recent edit first; ten undos in a row peel back ten edits in exactly the reverse order they were made. A pile of laundry, a stack of trays, and a Pringles can all share the identical underlying shape, even though only one of them is made of data.

Building `Stack` on Week 2's dynamic array

```
class Stack:
    def __init__(self):
        self._items = []          # Week 2's dynamic array, doubling under the hood

    def push(self, x):
        self._items.append(x)      # Week 2 Lecture 2/3: amortized O(1)

    def pop(self):
        if not self._items:
            raise IndexError("pop from an empty stack")
        return self._items.pop()   # removes and returns the LAST element

    def peek(self):
        if not self._items:
            raise IndexError("peek at an empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0
```

A stack is not a new kind of memory layout — it is a disciplined interface placed on top of the dynamic array already built and analyzed in Week 2. `push` maps directly onto `list.append`, and `pop` (with

no index argument) removes from the *end* of the underlying list, which Week 2 Lecture 4 established as the cheap position. The stack class itself adds no new storage mechanism; it only restricts *which* operations of the underlying list are exposed to the caller — no `insert(0, x)`, no indexing into the middle. That restriction is precisely what turns a general-purpose array into a stack.

Why `pop()` from the end, not the start

This design decision is not arbitrary — it is a direct consequence of Week 2 Lecture 4's analysis. Removing an element from the end of a dynamic array costs O(1), since no other element needs to move. Removing from the start costs O(n), since every remaining element must shift left to close the gap. A correct `Stack` implementation must therefore call `list.pop()` with no argument (removing the last element) and never `list.pop(0)` (removing the first), even though both are syntactically valid Python — only one of them preserves the O(1) cost a stack is expected to guarantee.

Cost of every stack operation

Every stack operation costs O(1), and every single one of those costs was already proven in a previous lecture, not newly asserted here: `push` inherits its amortized O(1) cost from Week 2's dynamic-array analysis; `pop()` is O(1) because it removes from the array's end, per Week 2 Lecture 4; `peek()` is O(1) because it is a single array index access, exactly as analyzed in Week 1 Lecture 1; and `is_empty()` is O(1) because Python lists track their length directly rather than counting elements. This is a deliberate teaching point: a stack has no new complexity of its own — it is a thin, disciplined interface over machinery already fully understood.

The call stack: a stack you have already used

```
def a():
    b()
    print("a resumes here")

def b():
    c()
    print("b resumes here")

def c():
    print("c runs")

a()
```

Every function call you have written since Week 1 has been using a stack implicitly: the *call stack*. Calling `a()` pushes a frame containing where to resume and its local variables. Inside `a()`, calling `b()` pushes a second frame on top of the first. Inside `b()`, calling `c()` pushes a third frame on top of both. `c()` finishes and returns first, popping its frame off the top — control returns to exactly where `b()` left off, which then finishes and pops its own frame, returning control to `a()`. This is Last In, First Out ordering, operating silently underneath every program this course has already written.

Tracing the call stack, frame by frame

```
Call a() → stack: [a]
  Call b() → stack: [a, b]
    Call c() → stack: [a, b, c]
    c() finishes, prints "c runs" → pop → stack: [a, b]
  b() resumes, prints "b resumes here" → pop → stack: [a]
a() resumes, prints "a resumes here" → pop → stack: []
```

Tracing the exact sequence: calling `a()` pushes frame `a`, leaving the stack `[a]`. Inside it, calling `b()` pushes frame `b` on top, giving `[a, b]`. Inside that, calling `c()` pushes frame `c`, giving `[a, b, c]`. `c()` prints its message and returns, popping its own frame — the stack shrinks back to `[a, b]`, and control resumes inside `b()` at exactly the line after the call to `c()`. This continues outward until the stack is empty and the program's entry call has fully returned. A `RecursionError` in Python is literally this stack overflowing its reserved size — too many frames pushed without enough pops in between, from a recursive call that never reaches its base case.

Application: checking balanced parentheses

A classic application makes the stack's LIFO property concrete: checking whether a string's brackets are balanced, such as `"(a + [b * (c - d)]) / e"`. The rule for correctness is that every closing bracket must match the most recently opened bracket that has not yet been closed. "Most recently opened, still unclosed" is precisely what sits on top of a stack at any moment — which is exactly why a stack is the natural structure for this problem, not merely a convenient one.

The algorithm

```
def is_balanced(s):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = Stack()
    for ch in s:
        if ch in "([{":
```

```
            stack.push(ch)
        elif ch in ")]}":
            if stack.is_empty() or stack.pop() != pairs[ch]:
                return False
    return stack.is_empty()
```

The algorithm scans the string once, left to right. Every opening bracket is pushed onto the stack. Every closing bracket triggers a pop: if the stack is empty (a closing bracket with nothing open to match) or the popped bracket does not correspond to this closing bracket, the string is unbalanced. After the scan, the string is balanced only if the stack is completely empty — no bracket was left open. This single pass costs O(n) for a string of length n, since every character triggers at most one push or one pop.

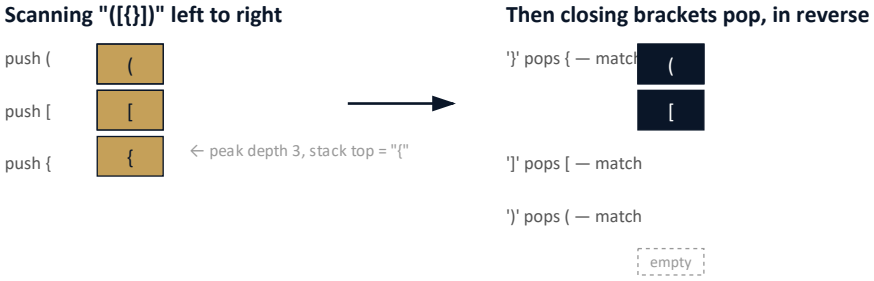
Tracing the algorithm on a real input

Tracing `"([)]"` character by character: the opening `(` is pushed, giving stack `[(]`. The opening `[` is pushed on top, giving `[(, []`. The closing `)` triggers a pop, which removes `[` — but `)` expects to match `(`, not `[`, so the function immediately returns `False`. This correctly identifies `"([)]"` as unbalanced, even though it superficially "looks" like it has matching bracket counts (one of each) — the stack catches that the *order* of closing is wrong, which a simple counting approach would miss entirely.

Tracing a correctly balanced input

Tracing the correctly nested `"([{}])"`: three opening brackets push in order, giving stack `[(, [, {]`. Then three closing brackets arrive in exactly the reverse order, each popping and matching correctly: `}` matches the just-pushed `{`, `]` matches `[`, and `)` matches `(`. The stack ends empty, correctly reporting the string as balanced. Notice the closing brackets had to arrive in exactly the mirror order of the opening ones — this is precisely why a stack, not a queue, is the right tool: a queue's FIFO ordering (next lecture) would check brackets in the *wrong* order for this problem.

The picture: stack growing and shrinking



Every close pops the most recently opened, still-unclosed bracket — the stack's top, by definition.

The stack genuinely grows and shrinks like a physical pile as the string is scanned — every opening bracket adds a box on top, every correctly matched closing bracket removes the top box, and the pile's height at any point in the scan tells you the current nesting depth.

MTech addition — why not just count brackets?

A tempting shortcut is to simply count opening and closing brackets of each type and check the counts match. This correctly catches simple mismatches like `"(()"`, where `(` outnumbers `)`. But it fails on `"([)]"`, which has exactly one of each of `(`, `)`, `[`, `]` — a pure counting approach would incorrectly report this as balanced, missing that the closing order is wrong. The stack-based algorithm succeeds precisely because it encodes *order*, via LIFO, not merely quantity — this is the structural reason a stack is the correct tool for this class of problem, not an incidental implementation convenience.

Exercise

As an exercise, extend `is_balanced` so that it ignores brackets appearing inside string literals — for example, `"a = '(unbalanced'"` should report `True`, since the unmatched `(` is inside quotes, not real code structure. Before running your extended version, trace it by hand on `"func(a, ')', b)"` to confirm it correctly treats the `)` inside the quoted string as ordinary text, not a real closing bracket.

Quiz — trace it yourself

Before checking the answer, trace the balanced-parentheses algorithm by hand on two inputs: `"{[()]}"` and `"{[(])}"`. Determine which one returns `True`, and for the one that returns `False`, identify the exact character at which the mismatch is first detected.

Quiz — answer

`"{[()]}"` is correctly and symmetrically nested — every closing bracket matches the most recently opened one in order, so the algorithm returns `True`. `"{[(])}"` fails: after pushing `{`, `[`, and `(` (stack `[{, [, (]`), the next character `]` triggers a pop, removing `(` — but `]` expects to match `[`, not `(`, so a mismatch is detected at this 5th character, and the function returns `False` immediately, without even examining the remaining characters.

Summary

A stack enforces Last In, First Out ordering by restricting access to only the top element, through `push` and `pop`. It is built directly on top of Week 2's dynamic array, and every operation's O(1) cost is inherited directly from that earlier analysis rather than newly asserted. The call stack, used silently since Week 1's very first function call, is a genuine instance of this same structure. Checking balanced parentheses works specifically because a stack encodes the *order* in which brackets were opened, not merely how many of each type appeared — a distinction a simple counting approach misses entirely. Next lecture turns to queues, the FIFO counterpart to today's LIFO stack, and uncovers a genuine trap: the obvious array-backed queue implementation silently inherits an O(n) cost from exactly the Week 2 analysis that made stacks cheap.